

NoboGhat

Technical Architecture & Development Blueprint

Version 1.0

This document defines the implementation blueprint for NoboGhat, describing the frontend, backend, database, APIs, authentication, development workflow, deployment strategy, and coding standards.

1. UI Wireframes (Conceptual)

Home

- ■ Hero
- ■ Search Route
- ■ Features
- ■ Footer

Dashboard

- ■ My Trips
- ■ Active Bookings
- ■ Notifications
- ■ Profile

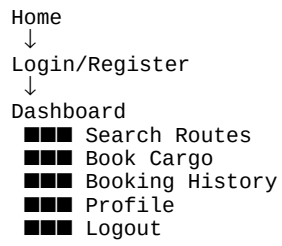
Boat Owner

- ■ Boats
- ■ Trips
- ■ Capacity
- ■ Bookings

Admin

- ■ Analytics
- ■ Users
- ■ Routes
- ■ Reports

2. Navigation Flow



3. Folder Structure

```
NoboGhat/  
  frontend/  
    assets/  
    css/  
    js/  
    pages/  
    index.html  
  backend/  
    controller/  
    service/  
    repository/  
    model/  
    dto/  
    config/  
    exception/  
  database/  
  docs/  
  README.md
```

4. REST API Design

```
POST /api/auth/register
POST /api/auth/login
GET  /api/users/profile
GET  /api/routes
GET  /api/trips
POST /api/bookings
GET  /api/bookings/{id}
DELETE /api/bookings/{id}
GET  /api/boats
POST /api/boats
GET  /api/admin/dashboard
```

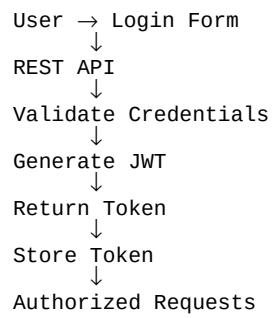
5. Database Schema

```
User(userId, name, phone, passwordHash, role)
Boat(boatId, ownerId, name, capacity)
Route(routeId, source, destination)
Trip(tripId, boatId, routeId, departureTime)
Booking(bookingId, userId, tripId, weight, status)
Payment(paymentId, bookingId, amount, status)
Notification(notificationId, userId, message)
```

6. Java Package Structure

```
com.noboghat
  ■■ controller
  ■■ service
  ■■ repository
  ■■ model
  ■■ dto
  ■■ config
  ■■ security
  ■■ exception
  ■■ util
```

7. Authentication Flow



8. Deployment Architecture

Browser
↓
Frontend (HTML/CSS/JS)
↓
Java Spring Boot API
↓
MySQL Database

Development:
Git → GitHub

Production:
Frontend → Vercel
Backend → Render/Railway
Database → MySQL/PostgreSQL

9. Coding Standards

- Follow MVC architecture.
- One class = one responsibility.
- Use meaningful names.
- Keep business logic in services.
- Validate all inputs.
- Write reusable methods.
- Document public classes.

10. Testing Strategy

Unit Testing
API Testing
Integration Testing
User Acceptance Testing
Regression Testing

11. Development Roadmap

Phase 0 Documentation
Phase 1 UI (HTML)
Phase 2 Styling (CSS)
Phase 3 JavaScript
Phase 4 Java Backend
Phase 5 Database
Phase 6 Authentication
Phase 7 Booking Engine
Phase 8 Admin Panel
Phase 9 Testing
Phase 10 Deployment

12. Long-term Vision

Future modules:

- Flood-aware route planning
- Weather integration
- GPS tracking
- Digital payments
- Warehouse management
- Analytics dashboard
- Android application
- AI-based cargo optimization